

CPE 663: Multilingual and Low Resource NLP

Major Assignment - III

Mini Transformer Benchmark Report

Introduction

The goal of this assignment is to understand the core concepts of the Transformer architecture by implementing a small Transformer encoder from scratch and evaluating it on a synthetic sequence classification task.

The model was executed manually with the PyTorch building block such as `nn.Linear`, `nn.Embedding`, `nn.LayerNorm`, and `nn.Dropout` instead of pretrained Transformer models or Transformer modules. This helps build a clearer understanding of how attention mechanisms, positional encoding, and encoder blocks work internally.

This is a binary sequence classification task in which the model should guess whether the first non-padding token will re-appear in the second half of the valid sequence or not.

The main objectives of this assignment are:

- understand token embeddings and positional encoding
- implement scaled dot-product attention
- implement multi-head self-attention
- build a Transformer encoder block
- train and evaluate a Transformer model
- compare multiple model variants using benchmarking

Method

Task Description

Each input is a padded token sequence with a maximum length of 20.

Vocabulary:

- PAD = 0
- A = 1
- B = 2
- C = 3
- D = 4

The actual sequence length is 6-20 and the shorter sequences are padded with the PAD tokens.

Label Definition

The label is defined as:

- label = 1 if the first non-padding token appears again in the second half of the non-padding sequence
- label = 0 otherwise

Example:

Sequence:

[A, C, B, D, A, PAD, PAD]

Valid sequence:

[A, C, B, D, A]

First token = A

Second half = [B, D, A]

Since A appears again, label = 1.

Model Architecture

The model is implemented using the following pipeline:

Input Tokens → Token Embedding → Positional Encoding → Transformer Encoder Block(s) → Mean Pooling → Classification Head → Binary Prediction

1. Token Embedding

The input token IDs are converted into dense vectors using:

```
nn.Embedding(vocab_size, d_model)
```

This allows the model to learn vector representations for each token.

2. Positional Encoding

Transformers do not necessarily know the order of tokens, so sinusoidal positional encoding is appended to token embeddings.

Sine values are applied to even dimensions and cosine values are applied to odd dimensions. This assigns an unique pattern to each token position to help in the model learning sequence order.

3. Multi-Head Self-Attention

The attention mechanism uses:

- Query (Q)
- Key (K)
- Value (V)

Scaled dot-product attention is calculated as:

$$\text{softmax}(QK^T/\sqrt{d_k})V$$

Multiple attention heads are used so the model can learn different types of token relationships simultaneously. Padding masks are applied to ensure PAD tokens are ignored during calculating attention.

4. Feed-Forward Network

A position-wise feed-forward network is applied after attention:

Linear → **ReLU** → **Linear**

This enhances representation learning after attention.

5. Residual Connection + Layer Normalization

Both attention and feed-forward sublayers are followed by residual connections and normalization of the layers to enable stable training.

6. Mean Pooling

Mean pooling over valid (non-padding) tokens is used to convert sequence output into a single vector for classification.

7. Classification Head

A final linear layer followed by sigmoid activation produces the binary output probability.

Experimental Setup

Hyperparameters

The following settings were used:

- Embedding dimension: 64
- Feed-forward dimension: 128
- Number of heads: 1 or 4
- Number of encoder layers: 1 or 2
- Dropout: 0.1
- Batch size: 32
- Learning rate: 0.001
- Epochs: 15
- Optimizer: Adam
- Loss function: Binary Cross Entropy Loss (BCELoss)

Reproducibility

Random seeds were fixed using:

```
torch.manual_seed(42)
```

along with NumPy and Python random seeds to ensure reproducibility of experiments.

Model Variants

Four model settings were compared.

Model	Positional Encoding	Heads	Layers
Model A	Yes	1	1
Model B	Yes	4	1
Model C	No	4	1
Model D	Yes	4	2

These settings were selected to meet the benchmark requirements:

- comparison with and without positional encoding
- comparison of different numbers of attention heads
- comparison of encoder depth

Results

Benchmark Table

Model	Positional Encoding	Heads	Layers	Validation Accuracy	Test Accuracy	Time	Parameters
Model A	Yes	1	1	0.8970	0.8970	0.02 min	33857
Model B	Yes	4	1	0.9270	0.9270	0.03 min	33857
Model C	No	4	1	0.8230	0.8230	0.03 min	33857
Model D	Yes	4	2	0.9390	0.9390	0.05 min	67329

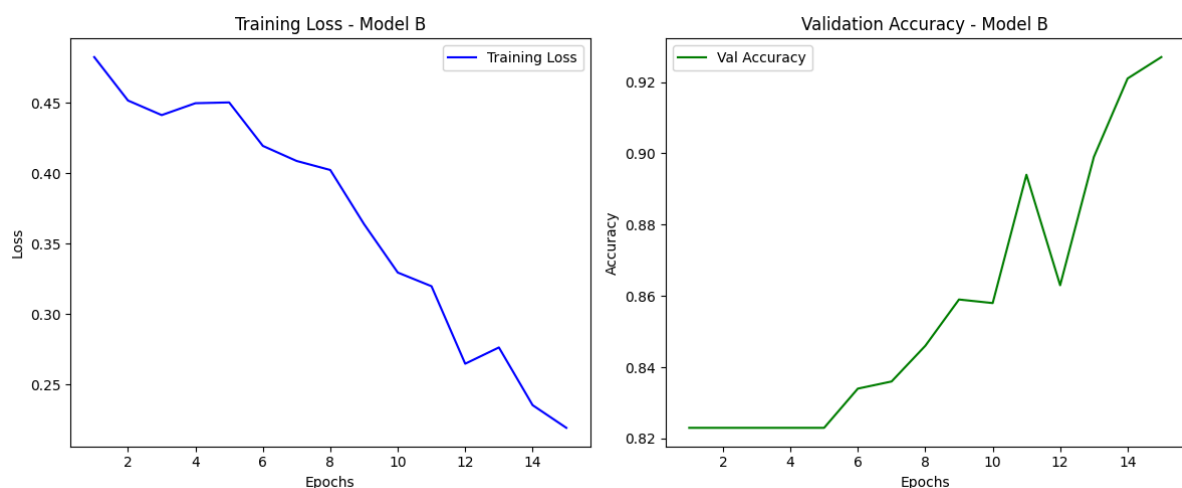
Training Curve

Model B training curves were plotted using:

- training loss by epoch
- validation accuracy by epoch

Training loss steadily declined which indicates the model learned effectively during training.

Validation accuracy improved significantly during the early epochs and then gradually stabilized at a high level. A small fluctuation was observed around epochs 10-12, which may be caused by batch variation or minor optimization instability. After that, the model maintained strong and stable validation performance, suggesting successful convergence without severe overfitting.



Discussion

Effect of Positional Encoding

Model C (without positional encoding) performed significantly worse than Model B.

- Model B: 0.9270
- Model C: 0.8230

This shows that positional encoding is important because Transformers do not naturally understand token order. Without positional information, the model struggles to identify sequence relationships effectively.

Effect of Multi-Head Attention

Model B (4 heads) outperformed Model A (1 head).

- Model A: 0.8970
- Model B: 0.9270

This implies that the model can capture more relations between tokens when there are multiple attention heads compared to single-head attention.

Effect of Additional Encoder Layer

Model D achieved the best performance.

- Model D: 0.9390

The introduction of a second encoder layer enhanced the quality of representation and classification. The training time and the number of parameters, however, also grew considerably.

This shows the tradeoff between performance and computational cost.

Parameter Count Analysis

The number of parameters in models A, B, and C is the same due to the fact that positional encoding is fixed and untrainable, and changing attention heads does not affect the total embedding dimensions.

The parameters of the model D have nearly doubled due to the inclusion of an additional encoder block.

Conclusion

This assignment provided hands-on insights into Transformer models internal works by building a mini Transformer encoder from scratch.

The experiments showed that:

- Positional encoding is essential for sequence understanding
- Multi-head attention enhances model performance
- Accuracy may further be enhanced by deeper encoder structures
- Stronger models often require more computation and parameters

Among all the variants, Model D was the best performing whereas Model C provided clear evidence of the significance of positional encoding.

Overall, this assignment helped strengthen understanding of attention mechanisms, Transformer encoder design, and experimental benchmarking in NLP systems.